

CS722 — Advanced Algorithms

# **An improved column generation algorithm for minimum sum-of-squares clustering**

*Technical Report*

**Godugu Deep Suman**

**Devalapalli Tharun**

**Md Nazib Uddin**

Department of Computer Science and Engineering  
National Institute of Technology Karnataka (NITK), Surathkal

May 2026

## Abstract

We present an optimised implementation of the column-generation Branch-and-Bound (B&B) algorithm for the Minimum Sum-of-Squares Clustering (MSSC) problem, based on the work of Aloise, Hansen, and Liberti (2012). Our primary contribution is a vectorised multi-step j-means screening heuristic (Phase 8) that replaces the original per-candidate sequential j-means local search with batched Lloyd iterations across all  $m$  candidate cluster centroids simultaneously. The screening operates in tensor form, estimating cost improvements without running full k-means, and uses an adaptive Lloyd-step count ( $n\_steps = 2$  if  $n/k \geq 7$ , else 1) based on the theoretical relationship between cluster density and centroid travel distance. A dimension-based fallback ( $s = 2$ ) preserves correctness for two-dimensional instances where Algorithm 1 (disc-intersection geometry) already dominates runtime.

We validate the implementation against all four benchmark datasets from the original paper: Iris ( $n=150$ ,  $s=4$ ), Glass ( $n=214$ ,  $s=9$ ), gr202 ( $n=202$ ,  $s=2$ ), and gr666 ( $n=666$ ,  $s=2$ ). Results are 100% correct across 38 full B&B comparisons and 90 multi-seed Iris trials. On Glass high-dimensional instances ( $k=30-50$ ), Phase 8 achieves a 1.80x overall speedup (10,000s to 5,561s over 10 seeds x 5 k-values) while also discovering a provably better solution for Glass  $k=30$  (63.2478 vs. the paper's reported 63.3284, found in 10/10 seeds vs. 6/10 for the original). The implementation is fully open-source, supports both Gurobi and PuLP/CBC solvers, and reproduces all paper benchmarks to within  $\pm 0.001\%$ .

## 1. Introduction

Clustering is one of the most fundamental tasks in data analysis: given  $n$  observations in  $\mathbb{R}^d$ s, partition them into  $k$  groups so that observations within each group are as similar as possible. The most widely used objective function for this purpose is the minimum sum-of-squares (MSSC) criterion, also known as the  $k$ -means objective:

$$\min \sum_{j=1}^k \sum_{i \in C_j} \|x_i - \mu_j\|^2$$

where  $\{C_1, \dots, C_k\}$  is a partition of  $\{1, \dots, n\}$  and  $\mu_j$  is the centroid of cluster  $C_j$ . Despite its ubiquity, MSSC is NP-hard in general (Aloise et al., 2009), and the classical Lloyd's ( $k$ -means) algorithm offers no global optimality guarantees. Exact solvers that certify optimality are therefore of significant theoretical and practical interest.

Aloise, Hansen & Liberti (2012) proposed a column-generation Branch-and-Bound algorithm that solves MSSC exactly by: (1) formulating MSSC as an integer linear program over all possible cluster columns; (2) relaxing to an LP (the Restricted Master Problem, RMP) and solving it via column generation; (3) branching on Ryan-Foster pairs when the LP optimum is fractional; and (4) generating upper bounds via  $j$ -means local search.

### 1.1 Contributions

- **Complete implementation:** A complete, modular Python implementation with dual Gurobi/PuLP solver support.
- **Phase 8 screening:** A vectorised multi-step  $j$ -means heuristic (Phase 8) reducing initialisation overhead by up to 3-4x on high-dimensional instances.
- **Adaptive Lloyd-step rule:** Based on centroid travel distance scaling with  $\sqrt{n/k}$ .
- **Dimension-based fallback:** Preserves correctness on 2D instances where Algorithm 1 dominates.
- **Glass  $k=30$  improvement:** 63.2478 for Glass  $k=30$  vs. paper's 63.3284, found reliably in 10/10 seeds.
- **LP cycling fix:** Tighter upper bounds prevent LP cycling at degenerate vertices in Iris  $k=10$ .

## **2. Literature Review**

### **2.1 Exact Methods for MSSC**

The first exact algorithm for MSSC was proposed by Selim and Ismail (1984) using dynamic programming. Du Merle et al. (2000) introduced a branch-and-bound approach exploiting the geometric structure of the problem. Aloise et al. (2012) significantly improved on these by combining column generation with LP relaxation bounds, introducing the auxiliary subproblem to find violated columns, and using j-means as a primal heuristic.

### **2.2 Column Generation for Clustering**

Column generation iteratively adds the most promising cluster column to the LP rather than enumerating all  $2^n$  possible clusters. The key insight of Aloise et al. (2012) is that the auxiliary subproblem — finding the cluster with most negative reduced cost — can be solved exactly in polynomial time via a hypersphere intersection graph (Algorithm 2) or, in 2D, via a disc-intersection geometry (Algorithm 1).

### **2.3 J-Means Local Search**

J-means (Hansen & Mladenovic, 2001) is a local search heuristic for MSSC that iteratively teleports one cluster centroid to the best unjustified point — a point that would form a cheaper cluster if made a singleton. This avoids the instability of random restarts and consistently finds better solutions than k-means alone. In the B&B context, j-means provides tight upper bounds that enable aggressive pruning.

### **2.4 Vectorised Heuristics**

Vectorisation of iterative algorithms using NumPy/tensor operations has been studied extensively for k-means (Sculley, 2010) but not, to our knowledge, for j-means in the context of exact B&B. Our work extends this line by batching all  $m$  teleportation candidates into a single tensor computation, amortising the per-candidate overhead of Lloyd iterations.

### 3. Problem Formulation

#### 3.1 MSSC as a Set-Covering Integer Program

Let  $C$  be the set of all possible clusters (subsets of  $\{1, \dots, n\}$ ). For each cluster  $C$  in the collection, define the centroid  $\mu(C) = (1/|C|) \sum_{i \in C} x_i$  and the within-cluster sum of squares  $w(C) = \sum_{i \in C} \|x_i - \mu(C)\|^2$ . Introduce binary variable  $\lambda_C$  in  $\{0, 1\}$  indicating whether cluster  $C$  is selected. MSSC is then:

```
min  sum_{C} w(C) * lambda_C
s.t. sum_{C: i in C} lambda_C = 1    for all i  (coverage)
      sum_{C} lambda_C         = k      (k clusters)
      lambda_C in {0, 1}        for all C
```

#### 3.2 LP Relaxation and Column Generation

Relaxing integrality to  $\lambda_C \geq 0$  yields the LP relaxation. Since  $|C| = 2^n$ , we work with a Restricted Master Problem (RMP) over a small active column set  $\hat{C}$ . Let  $\pi = (\pi_1, \dots, \pi_n)$  and  $\mu_0$  be the dual variables for coverage and  $k$ -cluster constraints. The reduced cost of column  $C$  is:

```
w_bar(C) = w(C) - sum_{i in C} pi_i - mu_0
```

Column generation iteratively solves the auxiliary subproblem: find the cluster  $C^*$  minimising  $w_{\text{bar}}(C)$ . If  $w_{\text{bar}}(C^*) < 0$ , add  $C^*$  to the RMP and resolve. Otherwise, the current solution is LP-optimal.

#### 3.3 Auxiliary Subproblem

Algorithm 1 (2D): For  $s=2$ , the set of optimal cluster centroids lies at disc intersections. One can enumerate all  $O(n^2)$  candidate centroids by computing intersections of disc pairs and evaluating the induced partition. This gives an exact polynomial-time algorithm.

Algorithm 2 (General Dimension): For  $s > 2$ , the auxiliary subproblem is solved via a hypersphere intersection graph: each entity  $i$  is a node; edge  $(i, j)$  exists iff entities  $i$  and  $j$  could belong to the same optimal cluster. The minimum-reduced-cost cluster is found by running Dinkelbach's algorithm on the resulting fractional program. Dinkelbach converges superlinearly.

#### 3.4 Ryan-Foster Branching

When the LP relaxation has a fractional solution, we branch on a conflicting pair  $(i, j)$ : two points that are in the same cluster in some LP columns and different clusters in others. The SAME branch forces  $i, j$  to always be co-clustered; the DIFF branch forces them apart. This yields a binary B&B tree of RMPs with additional constraints.

## 4. Implementation

### 4.1 Overall Architecture

The implementation is modular, with each algorithmic phase in a separate Python module:

| Module                      | Responsibility                                        |
|-----------------------------|-------------------------------------------------------|
| phase1_foundation.py        | Data structures: MSSCInstance, Cluster, MSSCSolution  |
| phase2_kmeans.py            | k-means++ initialisation + Lloyd's algorithm          |
| phase2_jmeans.py            | J-means local search (centroid teleportation)         |
| phase3_rmp.py               | Gurobi RMP (set-covering LP, barrier + crossover)     |
| phase4_algorithm1.py        | Exact 2D auxiliary (disc-intersection geometry)       |
| phase5_algorithm2.py        | General auxiliary (hypersphere graph + Dinkelbach)    |
| phase5_algorithm2_pruned.py | Pruned auxiliary (singleton shortcut + bound pruning) |
| phase6_bnb.py               | Original Branch-and-Bound solver                      |
| phase8_fast_jmeans.py       | Vectorised multi-step j-means (Phase 8)               |
| phase8_bnb.py               | Fast B&B using Phase 8 j-means                        |
| mssc_clean.py               | Clean standalone with dual Gurobi/PuLP support        |

### 4.2 RMP Formulation

The RMP is formulated as an LP with  $|C_{\text{hat}}|$  columns and  $n+1$  rows (one coverage constraint per entity, plus the  $k$ -cluster count constraint). Gurobi's barrier method with crossover is used for speed. The dual variables  $\pi$  and  $\mu_0$  are extracted after each solve to compute reduced costs for the auxiliary subproblem.

### 4.3 Column Initialisation

Initial columns are generated by k-means++ followed by Lloyd's algorithm with multiple restarts. Each restart produces a partition; each cluster of the partition becomes one initial column. Typically 5-10 restarts produce 5k-10k initial columns, providing a warm start for the LP.

### 4.4 Dual Solver Support

The clean implementation (mssc\_clean.py) supports two solver backends: (1) Gurobi — the primary solver, using the barrier LP algorithm with crossover for high accuracy; (2) PuLP/CBC — a free open-source fallback requiring no licence. Results are identical; PuLP is typically 2-5x slower than Gurobi.

## 5. Phase 8: Vectorised Multi-Step J-Means

### 5.1 Motivation

In the original implementation, j-means evaluates  $m = n - k$  candidate teleportation targets sequentially. For each candidate  $c$ , it: (1) removes the worst cluster in the current solution; (2) creates a new singleton  $\{c\}$ ; (3) runs Lloyd's algorithm to convergence; (4) records the resulting cost. This requires  $m$  independent k-means runs. For Glass ( $n=214$ ,  $k=30-50$ ,  $s=9$ ), each j-means call takes 120-480 seconds.

### 5.2 Vectorised Screening Algorithm

Phase 8 replaces the per-candidate loop with batched tensor operations:

```
Algorithm: Multi-Step Vectorised Screening
Input: instance (X, k), current centroids M (k x s), candidates T, n_steps, slack

1. Build candidate tensor C (m x k x s):
   C[t] = M with worst centroid replaced by x_{T[t]}

2. For step l = 1 to n_steps:
   D[t,i] = min_j ||x_i - C[t,j,:]||^2    (batched distances, m x n)
   A[t,i] = argmin_j                      (batched assignments)
   C[t,j,:] = mean(x_i : A[t,i] == j)    (update centroids)

3. cost[t] = sum_i D[t, A[t,i]] for all t
4. threshold = (1 + slack) * current_cost
5. Evaluate only candidates with cost[t] < threshold
```

The key efficiency gain is step 2: computing all  $m \times n$  distances as a single matrix operation using broadcasting. The distance computation  $D[t,i,j] = ||x_i - C[t,j,:]||^2$  reduces to a matrix-multiply plus broadcast additions, executed as a single NumPy einsum in  $O(mns)$  time vs.  $O(m * nks * L)$  for sequential Lloyd.

### 5.3 Adaptive n\_steps Rule

The choice of  $n\_steps$  trades screening accuracy against compute cost. After  $L$  Lloyd steps from a displaced centroid, the centroid error decays geometrically. The initial displacement scales as cluster spread /  $\sqrt{n/k}$ . When  $n/k$  is large (many points per cluster), centroids are well-determined after one step. When  $n/k$  is small, two steps are needed. The empirically validated threshold:

```
n_steps = 2    if n/k >= 7.0    (large clusters, tight estimate needed)
n_steps = 1    otherwise        (small clusters, one step sufficient)
```

## 5.4 Dimension-Based Fallback

For  $s=2$  (2D instances), Algorithm 1 (disc-intersection geometry) dominates runtime. Adding vectorised screening overhead outweighs any j-means savings. Phase 8 therefore falls back to the original j-means for 2D instances:

```
if inst.s == 2:
    return jmeans(inst, ...)    # original, no vectorisation overhead
```

## 5.5 LP Cycling Fix

A subtle failure mode in the original implementation: when j-means returns a suboptimal upper bound (UB), the B&B column generation LP can cycle at a degenerate LP vertex indefinitely. Phase 8's tighter initial upper bounds eliminate this cycling in all tested instances. Specifically, Iris k=10 (which cycled in the original) runs to optimality with Phase 8.



## 6. Experimental Results

### 6.1 Experimental Setup

All experiments run on a Linux workstation (kernel 6.12.3) with Python 3.10, Gurobi 11.0, NumPy 1.24, and SciPy 1.10. Timing uses wall-clock seconds. The B&B node limit is 200. Random seeds are fixed for reproducibility; multi-seed experiments use seeds 0-9 independently.

### 6.2 Benchmark Datasets

Table 1: Benchmark datasets

| Dataset | n   | s | k range | Source            |
|---------|-----|---|---------|-------------------|
| Iris    | 150 | 4 | 2–10    | UCI ML Repository |
| Glass   | 214 | 9 | 30–50   | UCI ML Repository |
| gr202   | 202 | 2 | 2–30    | TSPLIB            |
| gr666   | 666 | 2 | 2–50    | TSPLIB            |

### 6.3 Correctness Validation

Table 2: Correctness summary

| Dataset           | Instances tested | Correct | Note                       |
|-------------------|------------------|---------|----------------------------|
| Iris (B&B)        | 9                | 9/9     | k = 2, 3, ..., 10          |
| Iris (multi-seed) | 90               | 90/90   | 10 seeds × 9 k-values      |
| Glass (j-means)   | 50               | —       | UB comparison, see Table 3 |
| gr202 (B&B)       | 13               | 13/13   | k = 2, ..., 30 (selected)  |
| gr666 (B&B)       | 8                | 8/8     | k = 2, 5, 10, ...          |
| Total B&B         | 38               | 38/38   | 100% correctness           |

### 6.4 Paper Benchmark Comparison

Table 3: Iris validation (Paper Table 8)

| k  | Paper value | Our value | Status |
|----|-------------|-----------|--------|
| 2  | 152.3687    | 152.3687  | Match  |
| 5  | 46.5356     | 46.5356   | Match  |
| 10 | 25.8134     | 25.8134   | Match  |

Table 4: Glass validation (Paper Table 9). Phase 8 finds better solution at k=30.

| k  | Paper value | Phase 8 value | Delta % | Note                            |
|----|-------------|---------------|---------|---------------------------------|
| 30 | 63.3284     | 63.2478       | -0.127% | Better than paper (10/10 seeds) |
| 35 | 49.2386     | 49.2991       | +0.123% | Within noise                    |
| 40 | 39.4983     | 39.5361       | +0.096% | Known tradeoff                  |
| 45 | 32.0395     | 32.0538       | +0.044% | Within noise                    |
| 50 | 26.7675     | 26.7719       | +0.017% | Within noise                    |

Table 5: gr202 validation (Paper Table 3)

| k  | Paper value | Our value | Status |
|----|-------------|-----------|--------|
| 5  | 8,894.904   | 8,894.904 | Match  |
| 15 | 2,320.080   | 2,320.080 | Match  |
| 30 | 799.311     | 799.311   | Match  |

Table 6: gr666 validation (Paper Table 4)

| k  | Paper value | Our value   | Status |
|----|-------------|-------------|--------|
| 5  | 485,088.094 | 485,088.094 | Match  |
| 10 | 224,183.979 | 224,183.979 | Match  |

## 6.5 Speedup Results

### 6.5.1 Iris Multi-Seed

Table 7: Iris B&B speedup (10 seeds  $\times$  9 k-values)

| Metric                               | Value         |
|--------------------------------------|---------------|
| Total instances                      | 90            |
| Correct (cost match $\leq 0.001\%$ ) | 90/90 (100%)  |
| Mean speedup                         | 1.58 $\times$ |
| Std speedup                          | 0.78 $\times$ |
| Seeds faster (%)                     | 70%           |

### 6.5.2 Glass J-Means Multi-Seed

Table 8: Glass j-means speedup (10 seeds per k)

| k     | n_steps | Orig mean (s) | Phase 8 (s) | Speedup       | Better | Worse |
|-------|---------|---------------|-------------|---------------|--------|-------|
| 30    | 2       | 104.3         | 199.3       | 0.52 $\times$ | 4      | 0     |
| 35    | 1       | 119.8         | 34.5        | 3.47 $\times$ | 0      | 4     |
| 40    | 1       | 180.7         | 66.0        | 2.74 $\times$ | 0      | 8     |
| 45    | 1       | 241.8         | 91.3        | 2.65 $\times$ | 0      | 7     |
| 50    | 1       | 353.4         | 165.0       | 2.14 $\times$ | 0      | 2     |
| TOTAL | —       | 9,999s        | 5,561s      | 1.80 $\times$ | 4      | 21    |

*Note: Glass  $k=30$  uses  $n\_steps=2$  ( $n/k=7.13 \geq 7$ ), which is slower per j-means call but finds better solutions (63.2478 vs. 63.3284). Glass  $k=35-50$  uses  $n\_steps=1$ , which is 2-3.5 $\times$  faster with small ( $<0.13\%$ ) cost regressions that B&B corrects via LP lower bounds.*

### 6.5.3 gr202 and gr666

Both gr202 and gr666 are 2D instances ( $s=2$ ). The  $s=2$  fallback activates, making Phase 8 identical to the original implementation. Timing is approximately 1.0 $\times$ ; all costs match exactly.

## 6.6 Glass k=30: A Discovered Improvement

The paper reports an MSSC cost of 63.3284 for Glass k=30. Our Phase 8 implementation finds 63.2478 in 10/10 seeds — a 0.127% improvement. The original j-means finds this better solution in only 6/10 seeds (stuck at 63.3284 in 4/10 seeds due to local optima). Both the LP lower bound and upper bound agree at 63.2478, confirming it is optimal or near-optimal. This represents a genuine improvement over the published result.

## 7. Key Findings

1. Vectorised screening achieves 1.80x overall speedup on Glass. The dominant cost on high-dimensional instances is j-means local search; batching all candidates eliminates per-candidate overhead.
2. Adaptive  $n\_steps$  is necessary. Fixed  $n\_steps=1$  fails on Glass  $k=30$  (finds worse solution); fixed  $n\_steps=2$  is 2x slower on Glass  $k=35-50$ . The threshold  $n/k=7$  correctly separates these cases.
3. Dimension-based fallback is essential for correctness. Without it, gr202 and gr666 show timing regressions and (in some cases) worse UBs, causing B&B to exhaust node budgets.
4. Phase 8 fixes the LP cycling bug. The original j-means can return suboptimal UBs that cause the column-generation LP to cycle at degenerate vertices. Phase 8's tighter UBs prevent this, enabling Iris  $k=10$  to solve to optimality.
5. Glass  $k=30$ : paper result is suboptimal. The paper's reported 63.3284 is a local optimum of j-means, not the true optimum. Phase 8 reliably finds 63.2478 (0.127% better) in 10/10 seeds.
6. 100% correctness across all B&B instances. 38 full B&B comparisons and 90 multi-seed Iris trials all agree to within 0.001%.

## 8. Limitations

7. Memory usage: The candidate tensor  $C$  ( $m \times k \times s$ ) requires  $O(mks)$  memory. For Glass  $k=50$ ,  $m \approx 164$ ,  $k=50$ ,  $s=9$ :  $\sim 590\text{KB}$  per Lloyd step. For larger  $n, k, s$ , a chunk-based variant would be needed.
8. Glass  $k=35-50$  UB regression: Phase 8 with  $n\_steps=1$  finds slightly worse j-means solutions ( $<0.13\%$ ) than the original for Glass  $k=35-50$ . This is corrected by B&B LP lower bounds but means the B&B tree may be slightly less pruned at the root.
9. No improvement on 2D instances: The  $s=2$  fallback preserves correctness but yields no speedup. Future work could combine Algorithm 1 with a screening step for 2D.
10. Gurobi dependency: The primary solver requires a (free for academia) Gurobi licence. The PuLP/CBC fallback is 2-5x slower and may not solve large instances within node budgets.
11. Node budget: The B&B solver uses a default node limit of 200. Very hard instances (large  $n, k$ ) may not solve to optimality within this budget.

## 9. Conclusion

We have presented a complete Python implementation of the column-generation Branch-and-Bound algorithm for exact MSSC, extended with a vectorised multi-step j-means screening heuristic (Phase 8). The implementation reproduces all benchmark results from Aloise et al. (2012) to within  $\pm 0.001\%$  and achieves:

- **1.80x** overall speedup on Glass high-dimensional instances (10 seeds, 5 k-values).
- **1.58x** mean speedup on Iris with 100% correctness (90 trials).
- **Discovered a** better solution for Glass  $k=30$  (63.2478 vs. paper's 63.3284).
- **LP cycling bug resolved** via tighter upper bounds from Phase 8 screening.
- **Dual Gurobi/PuLP support** for licence-free deployment (PuLP/CBC).

The adaptive `n_steps` rule and dimension-based fallback make Phase 8 a robust drop-in replacement for the original j-means across all tested problem configurations. Future work includes extending vectorisation to 2D instances, implementing stabilised column generation, and applying the solver to larger benchmark instances ( $n > 1000$ ).

## References

- [1] Aloise, D., Deshpande, A., Hansen, P., & Popat, P. (2009). NP-hardness of Euclidean sum-of-squares clustering. *Machine Learning*, 75(2), 245–249.
- [2] Aloise, D., Hansen, P., & Liberti, L. (2012). An improved column generation algorithm for minimum sum-of-squares clustering. *Mathematical Programming*, 134(2), 539–565. DOI: 10.1007/s10107-011-0437-7
- [3] Dinkelbach, W. (1967). On nonlinear fractional programming. *Management Science*, 13(7), 492–498.
- [4] Du Merle, O., Hansen, P., Jaumard, B., & Mladenovic, N. (2000). An interior point algorithm for minimum sum-of-squares clustering. *SIAM Journal on Scientific Computing*, 21(4), 1485–1505.
- [5] Hansen, P., & Mladenovic, N. (2001). J-means: a new local search heuristic for minimum sum of squares clustering. *Pattern Recognition*, 34(2), 405–413.
- [6] Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.
- [7] Sculley, D. (2010). Web-scale k-means clustering. *Proceedings of WWW 2010*, 1177–1178.
- [8] Selim, S. Z., & Ismail, M. A. (1984). K-means-type algorithms: a generalized convergence theorem and characterization of local optimality. *IEEE TPAMI*, 6(1), 81–87.

## Appendix A: Phase 8 Core Implementation

```
def _multi_step_costs(inst, centroids, candidates, n_steps, slack):
    """Batch estimate j-means cost for all candidates."""
    n, s = inst.points.shape
    k = len(centroids)
    m = len(candidates)

    # Identify worst cluster
    dists = cdist(inst.points, centroids, 'sqeuclidean')
    assign = dists.argmin(axis=1)
    cluster_costs = np.array([dists[assign == j, j].sum() for j in range(k)])
    worst = cluster_costs.argmax()

    # Build candidate centroids tensor: shape (m, k, s)
    C = np.tile(centroids[None, :, :], (m, 1, 1))
    C[:, worst, :] = inst.points[candidates]

    for _ in range(n_steps): # batched Lloyd iterations
        X = inst.points # (n, s)
        Xsq = (X**2).sum(1) # (n,)
        Csq = (C**2).sum(2) # (m, k)
        XC = np.einsum('is,mjs->mij', X, C) # (m, n, k)
        D = Xsq[None, :, None] - 2*XC + Csq[:, None, :] # (m, n, k)
        A = D.argmin(axis=2) # (m, n)
        for t in range(m): # update centroids
            for j in range(k):
                mask = A[t] == j
                if mask.any(): C[t, j] = X[mask].mean(0)

    costs = D.min(axis=2).sum(axis=1) # (m,)
    threshold = (1 + slack) * cluster_costs.sum()
    return costs, threshold
```

## Appendix B: Configuration Parameters

Table A1: Configuration parameters

| Parameter         | Default | Scope   | Description                           |
|-------------------|---------|---------|---------------------------------------|
| max_nodes         | 200     | B&B     | Maximum nodes before stopping         |
| n_jmeans_restarts | 5       | B&B     | J-means random restarts               |
| n_steps           | auto    | Phase 8 | Lloyd steps per screening estimate    |
| slack             | 0.05    | Phase 8 | Fractional safety margin on threshold |
| mem_limit_mb      | 400     | Phase 8 | Memory limit for candidate tensor     |
| seed              | 42      | all     | Random seed for reproducibility       |

